QuillAudits

AUDIT REPORT

June 2026

For

LienFi

Table of Content

Executive Summary

Project Name	Lien Fi
Protocol Type	RWA, NFT Marketplace
Project URL	https://lienfi.com/
Overview	LienFi tokenizes US tax lien certificates as ERC721 NFTs on Base. A lien holder mints a LienNFT carrying a recorded base value, then either lists it on LienMarketplace for secondary trading where each sale charges a 2% fee, a 10% capital-gains fee on profit, and an optional seller surcharge, all routed to the treasury through a versioned FeeManager or locks it in LienVault, which is funded by liquidity providers and pays out principal plus accrued interest when the underlying lien is redeemed (with a 1% pre-sale profit share if the lien was never listed). The three core contracts (NFT, Vault, Marketplace) are UUPS-upgradeable, with admin and treasury as privileged roles.
Audit Scope	The scope of this Audit was to analyze the LienFi Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Minddeft-internal/lien-fi-smart-contracts/tree/prod
Branch	Prod
Contracts in Scope	LienVault.sol LienMarketplace.sol LienNFT.sol FeeManager.sol LienfiUUPSProxy.sol
Commit Hash	629ce62fa44b09865eb023eaa39017ef6fd287c7
Language	Solidity
Blockchain	Base
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	12th May 2026 - 19th May 2026

Updated Code Received	28th May 2026
Review 2	1st June 2026
Fixed In	8342208e87c3b08589f9c7c347e0c0c5b2a067a5

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity

Critical	0 (0%)
High	1 (5.26%)
Medium	4 (21.05%)
Low	10 (52.63%)
Informational	4 (22.22%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	**1**	**1**	**3**
Partially Resolved	0	0	0	0	0
Resolved	0	**1**	**3**	**9**	**1**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Capital Gains Fee Inflated by Seller Surcharge	High	Resolved
2	Signature Frontrunning Buyer's Purchase Can Be Stolen	Medium	Resolved
3	Old Signature Can Authorize a Re-listed NFT Sale	Medium	Resolved
4	USDC-Blocklisted Treasury Freezes All Fee-Bearing Transactions	Medium	Acknowledged
5	Cancelled pre-sale listing leaves a stale fee-version snapshot, causing redeeming holders to be charged an outdated profit-sharing rate	Medium	Resolved
6	setSellerSurcharge Does Not Verify Listing Is Active	Low	Resolved
7	Dead Code Unused _exists Function in LienNFT	Low	Resolved
8	One-Step Admin Transfer Risks Permanent Lockout	Low	Resolved
9	Missing SafeERC20 for Payment Token Transfers	Low	Resolved
10	Missing Zero Address Check for initialOwner in LienMarketplace	Low	Resolved

Issue No.	Issue Title	Severity	Status
11	Redundant Owner Check in listNFT	Low	Resolved
12	sellerSurcharge Not Included in Signed Quote Buyer Has No Payment Ceiling	Low	Resolved
13	Combined BPS Fees Not Validated sellerProceeds Can Underflow	Low	Acknowledged
14	Old Vault Retains VAULT_ROLE After Migration, Creating Permanent adminCancelListing Lock	Low	Resolved
15	FeeManager Migration Without Historical Config Backfill Permanently Blocks Lien Redemptions and New Listings	Low	Resolved
16	setMarketplace Does Not Migrate Active Listing State	Informational	Acknowledged
17	LienRedeemed Event Emits Gross Value Instead of Net Payout	Informational	Resolved
18	lienNFT Address Is Immutable in LienVault and LienMarketplace	Informational	Acknowledged
19	Three FeeConfig Fields Validated But Never Used in Fee Calculations	Informational	Acknowledged

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

Capital Gains Fee Inflated by Seller Surcharge

Resolved

Path

LienMarketplace.sol

Function Name

`_calculateCapitalGainsFee()`

Description

_collectFeesAndPayout passes totalAmount (= lienPrice + sellerSurcharge) to _calculateCapitalGainsFee. The surcharge is a treasury fee, not a capital gain, so including it inflates impliedInterest on every sale where surcharge > 0. costBasis is what the previous buyer paid (previousTotalAmount), so the real gain is currentLienPrice - costBasis. The code instead computes: (currentLienPrice + currentSurcharge) - costBasis, overcharging the buyer by exactly currentSurcharge on every sale.

Impact

Every buyer overpays capital gains fee by (surcharge * interestFeeBps / 10_000) on every sale. The overcharge goes to treasury and is not recoverable by the buyer.

Likelihood

High. Surcharge is non-zero in normal protocol operation, so every sale is affected.

Recommendation

Pass lienPrice instead of totalAmount to _calculateCapitalGainsFee.

Medium Severity Issues

Signature Frontrunning Buyer's Purchase Can Be Stolen

Resolved

Path

LienMarketplace.sol

Function Name

`buyNFT()`

Description

buyNFT accepts a backend-signed price signature as a parameter. The signature is bound to tokenId, lienPrice, maturity, deadline, baseValue, and feeVersion but NOT to the buyer's address (msg.sender). Any attacker copy the exact same parameters and call buyNFT ahead of the original buyer. Since the signature does not bind to any specific caller, the attacker's transaction succeeds and the original buyer's transaction reverts with listing inactive or signature already used.

Impact

The original buyer loses the NFT purchase to the frontrunner and wastes gas. The frontrunner acquires the NFT at the signed price with no extra effort. This completely breaks the intended buyer-specific purchase flow.

Likelihood

Medium. The mempool is public on almost all EVM chains and MEV bots routinely scan for extractable transactions. Front-running on the Base chain is highly improbable for typical market participants, though it remains theoretically possible.

Recommendation

Include msg.sender (buyer address) in the EIP-712 signature typehash so the signature is only valid for the intended buyer.

Medium Severity Issues

Old Signature Can Authorize a Relisted NFT Sale

Resolved

Path

LienMarketplace.sol

Function Name

`buyNFT()`

Description

The signature typehash covers (tokenId, lienPrice, maturity, deadline, baseValue, feeVersion) but does not include a listing nonce or seller address. usedSignatures only marks a signature consumed after a successful purchase. If a seller lists, a backend quote is issued, the seller then cancels and re-lists at different terms, the original quote is still valid and usable. Any party holding the old signed quote can purchase the NFT at the stale price as long as the deadline has not passed.

Impact

A buyer or attacker can use an old backend quote to buy a re-listed NFT at a price the seller no longer intends. The seller receives less than expected with no way to invalidate old quotes except waiting for deadline expiry.

Likelihood

Medium. Requires an old non-expired signature, which is realistic since backends sign quotes with future deadlines for user convenience.

Recommendation

Include a per-listing nonce in the typehash, incremented on every listNFT and cancelListing call so old quotes are automatically invalidated

Medium Severity Issues

USDC-Blocklisted Treasury Freezes All Fee Bearing Transactions

Acknowledged

Path

LienMarketplace.sol

Function Name

`buyNFT()`

Description

In both buyNFT and redeemLien fee paths, the treasury address receives the first USDC transfer with no fallback or try/catch. If Circle blocklists the treasury address, every call to transferFrom(buyer, treasury, ...) reverts. Since the fee transfer is the first interaction in the payment sequence, the entire transaction reverts atomically no purchase or redemption can
complete. There is no alternative recipient and no admin escape hatch to redirect fees to a non-blocklisted address mid-execution.

Impact

A blocklisted treasury completely halts all marketplace purchases and vault redemptions. The protocol becomes non-functional until the treasury is updated via updateTreasury, which itself requires the admin to act and a new transaction to be mined. During that window all users are DoS'd.

Likelihood

Low. USDC blocklisting of a protocol-controlled treasury is rare but not impossible, especially under regulatory pressure or an exploit scenario where the treasury is a compromised multisig.

Recommendation

Use a pull-payment pattern for fees: accumulate owed fees in a mapping and let the treasury or specific recipient claim them separately.

Lien Fi Team's Comment :

We can update the treasury wallet to resume the operations. Additionally, CIRCLE do not blacklist a protocol-controlled treasury wallet without major reason.

Medium Severity Issues

Cancelled pre-sale listing leaves a stale fee-version snapshot, causing redeeming holders to be charged an outdated profit-sharing rate.

Resolved

Path

LienMarketplace.sol

Function Name

`setSellerSurcharge()`

Description

The vault keeps a per-token fee-version snapshot in feeVersionByTokenId[tokenId] that determines which historical fee schedule a lien is taxed under at redemption. This mapping is written in exactly two places, setFeeVersion and forceFeeVersion, and is never reset anywhere in the codebase. During listing, _listNFT calls lienVault.setFeeVersion(tokenId, feeVersion) with the current fee version, which is always non-zero because both setFeeVersion and the listing entry points revert on a zero version. The redemption logic in _redeemLien contains a deliberate pre-sale fallback that reads if (feeVersion == 0 && isPreSale && address(feeManager) != address(0)) { feeVersion = feeManager.currentFeeVersion(); }, whose entire purpose is to ensure that a lien which was never purchased is taxed under the current schedule rather than a stale one. The correctness of this fallback rests on the assumption that a never-purchased lien always has feeVersionByTokenId == 0.

The cancel path breaks that assumption: _cancelListing only unlocks the NFT and sets isActive = false, and it never clears the vault's fee-version snapshot. Because isPreSale is derived from hasPurchasePriceByTokenId, which is set true only inside _buyNFT and never touched by cancel, a listed-then-cancelled lien remains isPreSale == true while carrying a non-zero feeVersionByTokenId. The fallback's feeVersion == 0 term is therefore false, the fallback is permanently dead for any lien that was ever listed, and the redemption applies the stale snapshot version. The stale path executes rather than reverting because FeeManager.createFeeConfig increments versions monotonically and retains every historical config in _feeConfigs, so getFeeConfig(staleVersion) succeeds and returns the outdated preSaleProfitSharingBps.

Attack Path

1. Fee version 1 is active with preSaleProfitSharingBps = 5000 (50%).
2. Alice owns pre-sale lien #8 (hasPurchasePriceByTokenId[8] == false, snapshot is 0).
3. Alice calls listNFT(8), which routes through _listNFT and calls lienVault.setFeeVersion(8, 1), setting feeVersionByTokenId[8] = 1. The NFT is locked in the vault.

4. Alice calls cancelListing(8). _cancelListing unlocks the NFT back to Alice and sets isActive = false, but never clears feeVersionByTokenId[8], which stays at 1. hasPurchasePriceByTokenId[8] is still false, so the lien is still considered pre-sale.
5. The admin publishes fee version 2 with preSaleProfitSharingBps = 0, making currentFeeVersion = 2.
6. An operator calls redeemLien(8, 2000) with stored RedemptionValue = 1000.
7. _redeemLien reads feeVersion = feeVersionByTokenId[8] = 1. Because it is non-zero, the feeVersion == 0 pre-sale fallback is skipped, so the current version 2 is never selected.
8. The redemption loads version 1's config via getFeeConfig(1) (which still exists, as old configs are retained), computes interestEarned = 2000 - 1000 = 1000, and charges platformFee = 1000 * 5000 / 10000 = 500 USDC to the treasury.
9. Had the snapshot been cleared on cancel, the fallback would have selected version 2 and charged 0. Alice is therefore overcharged 500 USDC under a schedule the protocol no longer applies. With the rates inverted (stale rate lower than current), the protocol under-collects instead.

Impact

A holder redeeming a previously-listed-and-cancelled pre-sale lien is taxed under whichever fee schedule was current at the moment of listing rather than the schedule in force at redemption, whenever the fee configuration changed in between. When the stale rate is higher than the current one, the holder is overcharged and loses the excess to the treasury; when the stale rate is lower, the protocol and treasury under-collect the fee they are owed. In both directions real value is misallocated to the wrong party, and because the corrupted snapshot is never cleared the discrepancy is deterministic and permanent for the life of the lien. An honest operator performing a routine redemption cannot avoid the wrong charge, so the defect manifests independently of any privileged misuse.

Recommendation

Clear the stale snapshot for un-purchased liens at cancel time by adding, in _cancelListing, a call such as if (!lienVault.hasPurchasePriceByTokenId(tokenId)) lienVault.clearFeeVersion(tokenId); after the unlock, which requires introducing a clearFeeVersion(tokenId) setter on the vault gated to MARKETPLACE_ROLE. A cleaner root-cause fix is to make the pre-sale path in _redeemLien always authoritative on the current version: whenever isPreSale is true, read feeManager.currentFeeVersion() unconditionally and ignore feeVersionByTokenId entirely, since a never-purchased lien should never be taxed under a snapshot that was only ever set by a listing event. This second approach also closes the related transfer-leak variant where the same per-token snapshots survive an off-market ERC721 transfer.

Low Severity Issues

setSellerSurcharge Does Not Verify Listing Is Active

Resolved

Path

LienMarketplace.sol

Function Name

`setSellerSurcharge()`

Description

setSellerSurcharge allows admin to set a surcharge on any tokenId without checking whether the listing is active. The surcharge can be written to inactive, cancelled, or nonexistent listings silently with no revert.

Recommendation

Add an isActive check before updating the surcharge

Low Severity Issues

Dead Code Unused _exists Function in LienNFT

Resolved

Path

LienNFT.sol

Function Name

`_exists()`

Description

_exists is an internal view function that wraps _ownerOf to check token existence. It is never called anywhere in the contract or inherited contracts.

Recommendation

Remove the function to reduce bytecode size and avoid confusion

Low Severity Issues

One Step Admin Transfer Risks Permanent Lockout

Resolved

Path

LienNFT.sol , LienVault.sol , LienMarketplace.sol

Function Name

`transferOwnership()`

Description

All three contracts implement a one-step admin transfer: DEFAULT_ADMIN_ROLE is granted to the new address and immediately revoked from msg.sender in the same transaction. If the new address is a typo, wrong chain address, or a contract that cannot call admin functions, admin access is permanently lost with no recovery mechanism.

Recommendation

Use a two-step transfer pattern: the current admin proposes a new admin, and the new admin must explicitly accept the role in a separate transaction.

Low Severity Issues

Missing SafeERC20 for Payment Token Transfers

Resolved

Path

LienVault.sol , LienMarketplace.sol

Function Name

`_redeemLien(), _collectFeesAndPayout()`

Description

All ERC20 transfers use raw require(paymentToken.transfer(...)) and require(paymentToken.transferFrom(...)) instead of OpenZeppelin SafeERC20. Tokens that do not return a bool (e.g. USDT on Ethereum mainnet) cause ABI decoding failure and revert. If the protocol changes the payment token or deploys on a chain where the token behaves differently, all transfers break.

Recommendation

Replace raw transfer calls with SafeERC20.safeTransfer and safeTransferFrom.

Low Severity Issues

Missing Zero Address Check for initialOwner in LienMarketplace

Resolved

Path

LienMarketplace.sol

Function Name

`initialize()`

Description

LienMarketplace.initialize does not validate that initialOwner is non-zero before granting DEFAULT_ADMIN_ROLE. LienVault and LienNFT both perform this check. If zero address is passed, DEFAULT_ADMIN_ROLE is granted to address(0) and no account can ever exercise admin functions.

Recommendation

Add the same check present in LienVault and LienNFT

Low Severity Issues

Redundant Owner Check in listNFT

Resolved

Path

LienMarketplace.sol , LienVault.sol

Function Name

`lockNFT(), listNFT()`

Description

listNFT checks ownerOfSafe(tokenId) == msg.sender, then calls lienVault.lockNFT which performs the same check again. The second check is redundant and wastes gas on every listing.

Recommendation

Remove the ownership check from lockNFT since the caller (LienMarketplace) already guarantees it, or remove it from listNFT and rely solely on lockNFT.

Low Severity Issues

sellerSurcharge Not Included in Signed Quote Buyer Has No Payment Ceiling

Resolved

Path

LienMarketplace.sol

Function Name

`_verifyPriceSignature(), buyNFT()`

Description

The EIP-712 signature covers (tokenId, lienPrice, maturity, deadline, baseValue, feeVersion) but not sellerSurcharge. The buyer's actual payment is totalAmount = lienPrice + sellerSurcharge. Since surcharge is absent from the signed data and can be changed by admin via setSellerSurcharge at any time, the buyer has no on-chain guarantee that the total they pay
matches the amount displayed or quoted. A buyer with a large PaymentToken allowance can silently overpay if the surcharge is raised after the quote is signed.

Recommendation

Include sellerSurcharge in the PRICE_TYPEHASH so the signed quote covers the full payment amount, or add a buyer-supplied maxTotalAmount parameter to buyNFT and revert if totalAmount exceeds it.

Low Severity Issues

Combined BPS Fees Not Validated sellerProceeds Can Underflow

Acknowledged

Path

LienMarketplace.sol , FeeManager.sol

Function Name

`_validateConfig(), _collectFeesAndPayout()`

Description

_validateConfig checks each BPS field independently against 10_000 but places no constraint on their combined sum. If the sum of fees exceeds totalAmount, this subtraction underflows. Solidity 0.8+ reverts on underflow so no funds are stolen, but the entire transaction reverts making the NFT unpurchasable until fee config is corrected.

Recommendation

Add a combined sum check in _validateConfig

LienFI Team's Comment

The underflow scenario requires an economically impossible fee configuration. For sellerProceeds to underflow, the combined fees must exceed totalAmount. Analysing the math:
platformFee is capped at lienPrice (when saleFeeBps = 10,000 = 100%)
capitalGainsFee is capped at impliedInterest which is always less than totalAmount
Even at maximum BPS for both, sellerProceeds only underflows when a lien sells at a gain AND, both fees are simultaneously 100%, Transaction will revert no harm to NFT and tokens

Low Severity Issues

Old Vault Retains VAULT_ROLE After Migration, Creating Permanent adminCancelListing Lock

Resolved

Path

LienMarketplace.sol , LienVault.sol

Description

if LienVault is upgraded to a new proxy address or replaced by a new deployment, the new vault address will not automatically receive VAULT_ROLE on the existing marketplace. Simultaneously, the old vault address retains VAULT_ROLE indefinitely because there is no function to revoke it. This may create a state where redemptions of vault-held NFTs revert because adminCancelListing is called by a contract that lacks the required role, or where a stale vault retains privileged marketplace access after migration.

Recommendation

Consider reviewing the design decision regarding vault upgrade. If upgrade is allowed then add appropriate access control in LienMarketplace to update the vault address and provide VAULT_ROLE.

Low Severity Issues

FeeManager Migration Without Historical Config Backfill Permanently Blocks Lien Redemptions and New Listings

Resolved

Path

LienMarketplace.sol , LienVault.sol

Description

When an admin migrates LienVault and LienMarketplace to a freshly deployed FeeManager that has currentFeeVersion = 0 (no fee configs yet populated), all previously purchased liens become permanently unredeemable and all new listings are immediately blocked.

Recommendation

Guard setFeeManager in both LienVault and LienMarketplace to reject a FeeManager that has not yet been populated with at least one fee config

setMarketplace Does Not Migrate Active Listing State

Acknowledged

Path

LienVault.sol

Function Name

`setMarketplace()`

Description

setMarketplace atomically revokes MARKETPLACE_ROLE from the old marketplace and grants it to the new one. NFTs actively listed on the old marketplace remain locked in LienVault with listings[tokenId].isActive == true on the old contract, which no longer has MARKETPLACE_ROLE. Those NFTs are stuck the old marketplace cannot call unlockNFT (role revoked) and the new marketplace has no record of the listings.

Recommendation

Before switching the marketplace, require all active listings to be cancelled, or implement a migration function that transfers listing state to the new marketplace contract.

LienFi Team's Comment

Before switching the marketplace, all active listings will be cancelled accordingly. This scenario only occurs if we update the proxy, not the implementation contract.

LienRedeemed Event Emits Gross Value Instead of Net Payout

Resolved

Path

LienVault.sol

Function Name

`_redeemLien()`

Description

The LienRedeemed event is emitted with amount = currentRedemptiveValue (the gross redemption amount), while the actual PaymentToken transferred to the owner is ownerPayout = currentRedemptiveValue - platformFee. When platformFee > 0, off-chain indexers and users reading the event will see an overstated payout amount.

Recommendation

Emit ownerPayout instead of currentRedemptiveValue, or add a separate platformFee field to the event so consumers can derive both value

lienNFT Address Is Immutable in LienVault and LienMarketplace

Acknowledged

Path

LienVault.sol

Description

LienNFT exposes a setLienVault(address) function at that allows its internal vault reference to be updated by the admin. However, neither LienVault nor LienMarketplace provides a corresponding setLienNFT() setter both contracts set lienNFT once in initialize() and have no mechanism to update it thereafter.

Recommendation

Add a setLienNFT(address) function (gated to DEFAULT_ADMIN_ROLE) in both LienVault and LienMarketplace to allow the NFT contract address to be updated without full redeployment. This symmetrizes the upgrade surface

Three FeeConfig Fields Validated But Never Used in Fee Calculations

Acknowledged

Path

FeeManager.sol

Description

FeeConfig contains six BPS fee fields. Three of them- saleServicer FeeBps, interestServicerFeeBps, and foreclosureFeeBps - are validated by _validateConfig to prevent values above BPS_DENOMINATOR and are included in the FeeConfigCreated event and hash, but are never read in any fee calculation anywhere in the protocol.

Recommendation

Either implement the fee logic for the three unused fields or remove them from FeeConfig.

LienFi Taem's Comment

These fees are added considering a future implementation

Formal Verification Specs

FeeManager

- treasury_never_zero — treasury() != 0 holds after every function call
- rule_access_only_owner_creates_config — non-owner call to createFeeConfig reverts
- rule_access_only_owner_updates_treasury — non-owner call to updateTreasury reverts
- rule_fee_version_monotone — currentFeeVersion increases by exactly 1 on createFeeConfig
- rule_fee_version_never_decreases — currentFeeVersion unchanged by any non-create call
- rule_treasury_zero_address_reverts — updateTreasury(0) reverts
- rule_treasury_updated_correctly — treasury() equals newTreasury after valid updateTreasury
- rule_bps_exceeds_denominator_reverts — createFeeConfig reverts when any bps field > 10_000
- rule_bps_within_bounds_after_create — all stored bps fields are <= 10_000 after createFeeConfig
- rule_get_config_unknown_reverts — getFeeConfig reverts when feeScheduleHash for version is zero
- rule_get_config_known_succeeds — getFeeConfig does not revert for a version just created
- rule_fee_hash_non_zero_after_create — feeScheduleHash is non-zero after createFeeConfig

LienNFT

- next_token_id_at_least_one — nextTokenId == 0 OR nextTokenId >= 1 (trivially true; real property in rule below)
- rule_next_token_id_once_set_stays_positive — nextTokenId never drops below 1 once initialized

- rule_next_token_id_monotone — nextTokenId increases by exactly 1 after mintLien
- rule_next_token_id_never_decreases — nextTokenId never decreases from non-mint operations
- rule_access_only_minter_can_mint — non-MINTER_ROLE call to mintLien reverts
- rule_access_only_vault_updates_redemption_value — non-VAULT_ROLE call to updateRedemptionValue reverts
- rule_access_only_admin_sets_vault — non-DEFAULT_ADMIN_ROLE call to setLienVault reverts
- rule_access_only_admin_adds_minter — non-DEFAULT_ADMIN_ROLE call to addMinter reverts
- rule_access_only_admin_updates_uri — non-DEFAULT_ADMIN_ROLE call to updateTokenURI reverts
- rule_listed_status_set_correctly — isListed reflects status passed to setListedStatus
- rule_burn_clears_owner — ownerOfSafe returns address(0) after burnLien Revert Conditions
- rule_set_vault_zero_reverts — setLienVault(0) reverts
- rule_mint_zero_recipient_reverts — mintLien to address(0) reverts
- rule_transfer_ownership_zero_reverts — transferOwnership(0) reverts

LienVault

- rule_access_only_marketplace_lock — non-MARKETPLACE_ROLE call to lockNFT reverts
- rule_access_only_marketplace_unlock — non-MARKETPLACE_ROLE call to unlockNFT reverts
- rule_access_only_marketplace_set_fee_version — non-MARKETPLACE_ROLE call to setFeeVersion reverts
- rule_access_only_marketplace_set_purchase_price — non-MARKETPLACE_ROLE call to setPurchasePrice reverts
- rule_access_only_operator_redeem — non-OPERATOR and non-DEFAULT_ADMIN call to redeemLien reverts

- rule_access_only_admin_set_marketplace — non-DEFAULT_ADMIN call to setMarketplace reverts
- rule_access_only_admin_set_fee_manager — non-DEFAULT_ADMIN call to setFeeManager reverts
- rule_access_only_admin_remove_liquidity — non-DEFAULT_ADMIN call to removeLiquidity reverts
- rule_access_only_admin_force_fee_version — non-DEFAULT_ADMIN call to forceFeeVersion reverts
- rule_access_only_guardian_pause — non-GUARDIAN and non-DEFAULT_ADMIN call to pause reverts
- rule_access_only_admin_unpause — non-DEFAULT_ADMIN call to unpause reverts
- rule_pause_blocks_lock — lockNFT reverts when paused
- rule_pause_blocks_unlock — unlockNFT reverts when paused
- rule_pause_blocks_redeem — redeemLien reverts when paused
- rule_has_purchase_price_monotone — hasPurchasePriceByTokenId never goes from true to false
- rule_set_purchase_price_updates_state — purchasePriceByTokenId and hasPurchasePriceByTokenId both correct after setPurchasePrice
- rule_force_fee_version_zero_reverts — forceFeeVersion(0) reverts
- rule_set_marketplace_zero_reverts — setMarketplace(0) reverts

LienMarketplace

- rule_pause_blocks_list — listNFT reverts when paused
- rule_pause_blocks_buy — buyNFT reverts when paused
- rule_pause_blocks_cancel — cancelListing reverts when paused
- rule_access_admin_cancel_vault_only — non-VAULT_ROLE call to adminCancelListing reverts
- rule_access_only_admin_set_fee_manager — non-DEFAULT_ADMIN call to setFeeManager reverts

- rule_access_only_operator_set_price_signer — non-OPERATOR and non-DEFAULT_ADMIN call to setPriceSigner reverts
- rule_access_only_admin_set_surcharge — non-DEFAULT_ADMIN call to setSellerSurcharge reverts
- rule_access_only_guardian_pause — non-GUARDIAN and non-DEFAULT_ADMIN call to pause reverts
- rule_access_only_admin_unpause — non-DEFAULT_ADMIN call to unpause reverts
- rule_listing_inactive_after_cancel — listing.isActive is false after cancelListing
- rule_cancel_only_by_seller — cancelListing by non-seller reverts
- rule_cancel_inactive_listing_reverts — cancelListing on inactive listing reverts
- rule_buy_inactive_listing_reverts — buyNFT on inactive listing reverts
- rule_admin_cancel_deactivates_listing — adminCancelListing sets listing.isActive to false
- rule_has_purchase_price_monotone — hasPurchasePriceByTokenId never goes from true to false
- rule_get_seller_raw_returns_zero_when_inactive — getListingSellerRaw returns address(0) when listing inactive
- rule_get_seller_returns_zero_when_no_purchase_price — getListingSeller returns address(0) when vault has no purchase price
- rule_set_price_signer_zero_reverts — setPriceSigner(0) reverts
- rule_set_fee_manager_zero_reverts — setFeeManager(0) reverts

Functional Tests

Some of the tests performed are mentioned below:

FeeManager

- constructor reverts if initialTreasury is zero address
- constructor stores treasury correctly when valid
- constructor sets owner correctly via Ownable2Step
- createFeeConfig reverts if caller is not owner
- createFeeConfig reverts if any single bps field exceeds 10_000
- createFeeConfig stores all six bps fields correctly
- createFeeConfig increments currentFeeVersion by exactly 1
- createFeeConfig stores a non-zero feeScheduleHash for the new version
- createFeeConfig emits CurrentFeeVersionUpdated with old and new version
- updateTreasury reverts if caller is not owner
- updateTreasury reverts if newTreasury is zero address
- updateTreasury stores new treasury address
- updateTreasury emits TreasuryUpdated with old and new address
- getFeeConfig reverts for a version whose feeScheduleHash is zero
- getFeeConfig returns correct config for a created version
- feeScheduleHash reverts for a version whose hash is zero
- feeScheduleHash returns correct hash for a created version

LienNFT

- constructor calls _disableInitializers so implementation cannot be initialized
- initialize can only run once

- initialize reverts if initialOwner is zero address
- initialize sets name to Tax Liens NFT and symbol to Lien-NFT
- initialize grants DEFAULT_ADMIN_ROLE to initialOwner
- initialize grants MINTER_ROLE to initialOwner
- initialize sets nextTokenId to 1
- transferOwnership reverts if caller lacks DEFAULT_ADMIN_ROLE
- transferOwnership reverts if newOwner is zero address
- transferOwnership grants DEFAULT_ADMIN_ROLE to newOwner
- transferOwnership revokes DEFAULT_ADMIN_ROLE from msg.sender
- setLienVault reverts if caller lacks DEFAULT_ADMIN_ROLE
- setLienVault reverts if vault is zero address
- setLienVault stores new vault address
- setLienVault grants VAULT_ROLE to new vault
- setLienVault revokes VAULT_ROLE from old vault when updating
- setLienVault emits UpdatedVault event
- addMinter reverts if caller lacks DEFAULT_ADMIN_ROLE
- addMinter reverts if account is zero address
- addMinter grants MINTER_ROLE to account
- addMinter emits MinterAdded event
- removeMinter reverts if caller lacks DEFAULT_ADMIN_ROLE
- removeMinter revokes MINTER_ROLE from account
- removeMinter emits MinterRemoved event
- mintLien reverts if caller lacks MINTER_ROLE
- mintLien mints token to recipient with correct tokenId

- mintLien increments nextTokenId by 1
- mintLien stores all LienDetails fields correctly
- mintLien sets token URI from IpfsMetadataHash
- mintLien emits lienMinted event with correct fields
- bulkMintLiens reverts if caller lacks MINTER_ROLE
- bulkMintLiens reverts if recipient is zero address
- bulkMintLiens reverts if array length is zero
- bulkMintLiens reverts if array length exceeds 100
- bulkMintLiens mints correct number of tokens to recipient
- bulkMintLiens returns correct array of token IDs
- bulkMintLiens emits BulkMintCompleted with correct start and end IDs
- updateTokenURI reverts if caller lacks DEFAULT_ADMIN_ROLE
- updateTokenURI reverts if token does not exist
- updateTokenURI stores new URI for the token
- bulkUpdateTokenURIs reverts if tokenIds and newURIs lengths differ
- bulkUpdateTokenURIs updates each token URI correctly
- updateRedemptionValue reverts if caller lacks VAULT_ROLE
- updateRedemptionValue reverts if token does not exist
- updateRedemptionValue stores new RedemptionValue in lienDetails
- updateRedemptionValue emits RedemptionValueUpdated event
- setRedemptionType reverts if caller lacks VAULT_ROLE
- setRedemptionType reverts if token does not exist
- setRedemptionType stores new RedemptionType in lienDetails
- setRedemptionType emits RedemptionTypeUpdated event

- setListedStatus reverts if caller lacks VAULT_ROLE
- setListedStatus sets isListed to true for the token
- setListedStatus sets isListed to false for the token
- setListedStatus emits LienStatusUpdated event
- burnLien reverts if caller lacks VAULT_ROLE
- burnLien reverts if token does not exist
- burnLien burns the token so ownerOf reverts afterward
- burnLien emits LienRedeemed event with stored RedemptionValue and RedemptionType
- getRedemptionValue returns stored RedemptionValue for token
- lienDetails returns correct full struct for token
- getBulkLienDetails returns correct structs for an array of token IDs
- ownerOfSafe returns owner address for existing token
- ownerOfSafe returns zero address for non-existent token
- upgradeToAndCall reverts if caller lacks DEFAULT_ADMIN_ROLE
- upgradeToAndCall allows admin to upgrade implementation
- _exists — dead internal function never called, should be removed
- transferOwnership — one-step admin transfer risks permanent lockout

LienVault

- constructor calls _disableInitializers so implementation cannot be initialized
- initialize can only run once
- initialize reverts if _lienNFT is zero address
- initialize reverts if initialOwner is zero address
- initialize reverts if _paymentToken is zero address
- initialize reverts if _emergencyGuardian is zero address

- initialize reverts if _operator is zero address
- initialize grants DEFAULT_ADMIN_ROLE to initialOwner
- initialize grants GUARDIAN_ROLE to emergencyGuardian
- initialize grants OPERATOR_ROLE to operator
- initialize sets lienNFT address correctly
- initialize sets paymentToken address correctly
- initialize sets marketplace to zero address initially
- transferOwnership reverts if caller lacks DEFAULT_ADMIN_ROLE
- transferOwnership reverts if newOwner is zero address
- transferOwnership grants DEFAULT_ADMIN_ROLE to newOwner
- transferOwnership revokes DEFAULT_ADMIN_ROLE from msg.sender
- pause reverts if caller lacks GUARDIAN_ROLE or DEFAULT_ADMIN_ROLE
- pause succeeds for GUARDIAN_ROLE
- pause succeeds for DEFAULT_ADMIN_ROLE
- unpause reverts if caller lacks DEFAULT_ADMIN_ROLE
- unpause reverts if caller has only GUARDIAN_ROLE
- unpause succeeds for DEFAULT_ADMIN_ROLE
- setMarketplace reverts if caller lacks DEFAULT_ADMIN_ROLE
- setMarketplace reverts if address is zero
- setMarketplace stores new marketplace address
- setMarketplace grants MARKETPLACE_ROLE to new marketplace
- setMarketplace revokes MARKETPLACE_ROLE from old marketplace when updating
- setFeeManager reverts if caller lacks DEFAULT_ADMIN_ROLE
- setFeeManager reverts if address is zero

- setFeeManager stores new feeManager address
- setFeeManager emits FeeManagerUpdated event
- setFeeVersion reverts if caller lacks MARKETPLACE_ROLE
- setFeeVersion reverts if feeVersion is zero
- setFeeVersion reverts if token is not listed (isListed == false)
- setFeeVersion stores fee version for the token
- setFeeVersion emits FeeVersionAssigned event
- setPurchasePrice reverts if caller lacks MARKETPLACE_ROLE
- setPurchasePrice stores purchase price for the token
- setPurchasePrice sets hasPurchasePriceByTokenId to true
- setPurchasePrice emits PurchasePriceUpdated event
- forceFeeVersion reverts if caller lacks DEFAULT_ADMIN_ROL
- forceFeeVersion reverts if feeVersion is zero
- forceFeeVersion overrides feeVersionByTokenId for the token
- forceFeeVersion emits FeeVersionForced with previous and new version
- lockNFT reverts if caller lacks MARKETPLACE_ROLE
- lockNFT reverts when paused
- lockNFT reverts if provided owner does not match token owner
- lockNFT transfers NFT from owner to vault
- lockNFT sets isListed to true on LienNFT
- lockNFT emits NFTLocked event
- unlockNFT reverts if caller lacks MARKETPLACE_ROLE
- unlockNFT reverts when paused
- unlockNFT reverts if NFT is not owned by vault

- unlockNFT transfers NFT from vault to newOwner
- unlockNFT sets isListed to false on LienNFT
- unlockNFT emits NFTUnlocked event
- redeemLien reverts if caller lacks OPERATOR_ROLE and DEFAULT_ADMIN_ROLE
- redeemLien reverts when paused
- redeemLien reverts if NFT is in vault but isListed is false
- redeemLien reverts if listedPayoutRecipient resolves to zero address
- redeemLien reverts if NFT is not in vault and vault is not approved
- redeemLien reverts if vault has insufficient USDC balance
- redeemLien reverts if token has already been redeemed (RedemptionType set)
- redeemLien with isPreSale true uses preSaleProfitSharingBps for fee
- redeemLien with isPreSale false uses interestFeeBps on purchase price delta
- redeemLien with currentRedemptiveValue <= costBasis charges zero fee
- redeemLien transfers platformFee to treasury when fee > 0
- redeemLien transfers ownerPayout to payoutRecipient
- redeemLien calls burnLien on LienNFT after payouts
- redeemLien sets RedemptionType to Early when before maturity
- redeemLien sets RedemptionType to Expired when after maturity
- redeemLien emits LienRedeemed event
- redeemLien calls adminCancelListing when NFT is in vault (listed)
- redeemLien calls setListedStatus false when NFT is in vault
- addLiquidity reverts if amount is zero
- addLiquidity transfers USDC from caller to vault
- addLiquidity emits LiquidityAdded event

- removeLiquidity reverts if caller lacks DEFAULT_ADMIN_ROLE
- removeLiquidity reverts if to is zero address
- removeLiquidity reverts if amount is zero
- removeLiquidity reverts if vault balance is insufficient
- removeLiquidity transfers USDC to recipient
- removeLiquidity emits LiquidityRemoved event
- removeFullLiquidity reverts if caller lacks DEFAULT_ADMIN_ROLE
- removeFullLiquidity reverts if to is zero address
- removeFullLiquidity transfers entire vault balance to recipient
- removeFullLiquidity emits FullLiquidityRemoved event
- _redeemLien — treasury USDC blocklist DoS, no fallback recipient
- transferOwnership — one-step admin transfer risks permanent lockout
- _redeemLien — raw transfer instead of SafeERC20
- _redeemLien — LienRedeemed event emits gross value not net ownerPayout

LienMarketplace

- constructor calls _disableInitializers so implementation cannot be initialized
- initialize can only run once
- initialize reverts if lienNFT is zero address
- initialize reverts if paymentToken is not a contract
- initialize reverts if lienVault is zero address
- initialize grants DEFAULT_ADMIN_ROLE to initialOwner
- initialize grants GUARDIAN_ROLE to guardian

- initialize grants OPERATOR_ROLE to operator
- initialize sets priceSigner correctly
- initialize sets feeManager correctly
- pause reverts if caller lacks GUARDIAN_ROLE or DEFAULT_ADMIN_ROLE
- pause succeeds for GUARDIAN_ROLE
- pause succeeds for DEFAULT_ADMIN_ROLE
- unpause reverts if caller lacks DEFAULT_ADMIN_ROLE
- setFeeManager reverts if caller lacks DEFAULT_ADMIN_ROLE
- setFeeManager reverts if address is zero
- setFeeManager stores new feeManager and emits FeeManagerUpdated
- setPriceSigner reverts if caller lacks OPERATOR_ROLE and DEFAULT_ADMIN_ROLE
- setPriceSigner reverts if address is zero
- setPriceSigner stores new signer and emits PriceSignerUpdated
- setSellerSurcharge reverts if caller lacks DEFAULT_ADMIN_ROLE
- setSellerSurcharge stores surcharge value for the tokenId
- setSellerSurcharge emits SellerSurchargeUpdated event
- transferOwnership reverts if caller lacks DEFAULT_ADMIN_ROLE
- transferOwnership reverts if newOwner is zero address
- transferOwnership grants DEFAULT_ADMIN_ROLE to newOwner
- transferOwnership revokes DEFAULT_ADMIN_ROLE from msg.sender
- listNFT reverts when paused
- listNFT reverts if token is already listed
- listNFT reverts if caller is not token owner
- listNFT reverts if feeManager is zero address

- initialize grants OPERATOR_ROLE to operator
- listNFT reverts if currentFeeVersion is zero
- listNFT locks NFT into vault via lockNFT
- listNFT records listing with correct seller, surcharge, and isActive true
- listNFT emits LienNFTListed event
- buyNFT reverts when paused
- buyNFT reverts if listing is not active
- buyNFT reverts if feeVersion does not match currentFeeVersion
- buyNFT reverts if maturity param does not match lienDetails.Maturity
- buyNFT reverts if signature is expired
- buyNFT reverts if signature is from wrong signer
- buyNFT reverts if same signature is replayed
- buyNFT reverts if buyer has insufficient PaymentToken allowance
- buyNFT transfers platformFee to treasury
- buyNFT transfers capitalGainsFee to treasury
- buyNFT transfers sellerSurcharge to treasury
- buyNFT transfers sellerProceeds to seller
- buyNFT calls setFeeVersion on vault with correct version
- buyNFT calls setPurchasePrice on vault with totalAmount
- buyNFT calls unlockNFT to transfer NFT to buyer
- buyNFT sets listing.isActive to false
- buyNFT sets listing.buyer to msg.sender
- buyNFT sets listing.price to lienPrice
- buyNFT emits LienNFTPurchased event

- ✓ cancelListing reverts when paused
- ✓ cancelListing reverts if listing is not active
- ✓ cancelListing reverts if caller is not the listing seller
- ✓ cancelListing calls unlockNFT to return NFT to seller
- ✓ cancelListing sets listing.isActive to false
- ✓ cancelListing emits ListingCancelled event
- ✓ adminCancelListing reverts if caller lacks VAULT_ROLE
- ✓ adminCancelListing sets listing.isActive to false
- ✓ adminCancelListing emits ListingCancelled event
- ✓ getListingSeller returns zero address if listing is inactive
- ✓ getListingSeller returns zero address if hasPurchasePriceByTokenId is false
- ✓ getListingSeller returns seller address if listing is active and purchase price set
- ✓ getListingSellerRaw returns zero address if listing is inactive
- ✓ getListingSellerRaw returns seller address if listing is active regardless of purchase price
- ✓ _collectFeesAndPayout — totalAmount passed to _calculateCapitalGainsFee inflates capital gains fee by sellerSurcharge
- ✓ buyNFT transfers sellerSurcharge to treasury
- ✓ buyNFT — signature not bound to buyer address, frontrunning possible
- ✓ verifyPriceSignature — no listing nonce, old signature valid after cancel/relist
- ✓ buyNFT calls setPurchasePrice on vault with totalAmount
- ✓ _collectFeesAndPayout — treasury USDC blocklist DoS, no fallback recipient
- ✓ setSellerSurcharge — no isActive check allows write to inactive listing
- ✓ transferOwnership — one-step admin transfer risks permanent lockout

- initialize — missing zero address check for initialOwner
- listNFT — redundant owner check duplicated in lockNFT
- buyNFT — sellerSurcharge absent from signed quote, no buyer payment ceiling
- buyNFT — CEI violation, listing.isActive = false written after all external calls
- adminCancelListing — does not call setListedStatus false on LienNFT

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
OpenZeppelin Upgradeable (Initializable, AccessControlUpgradeable, ReentrancyGuardUpgradeable, PausableUpgradeable, ERC721URIStorageUpgradeable, EIP712Upgradeable)	Library / Base Contracts	Provide initializer pattern, role-based access control, reentrancy mutex, pause switch, ERC721 storage, and EIP-712 typed-data hashing for the three upgradeable contracts	Audited OZ implementations behave per spec; storage layout preserved across upgrades; modules initialized exactly once	Storage collision or gap mismanagement on upgrade; module left un-initialized; mutex/pause logic relied on but bypassable via trusted roles
UUPSUpgradeable + ERC1967Proxy (via LienfiUUPSProxy)	Base Contract	Delegatecall proxy holding all state for LienNFT, LienVault, LienMarketplace; upgrade gated by _authorizeUpgrade _authorizeUpgrade	Implementation slot only writable through _authorizeUpgrade (DEFAULT_ADMIN_ROLE)	Malicious or incorrect upgrade bricks or drains proxy state; uninitialized implementation; admin key compromise = total control

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Ownable2Step (OpenZeppelin)	Base Contract	Two-step ownership for FeeManager (fee config + treasury changes)	Owner is trusted and competent; new owner explicitly accepts ownership	Owner key compromise → arbitrary fee schedule and treasury redirection affecting all settlements
ECDSA (OpenZeppelin)	Library	Recovers priceSigner from the EIP-712 BuyPrice signature inside buyNFT	Recovers signer correctly; rejects malleable signatures (OZ v5 behavior)	Library bug would allow signature forgery / price spoofing
IERC20 / IERC20Metadata (OpenZeppelin)	Interface	Defines payment-token transfer and decimals interface used by vault and marketplace	Correct interface definition matching ERC20 standard	N/A (interface only)
Payment Token USDC on Base	External ERC20 Contract	Settlement currency for buys, all fees, redemption payouts, and vault liquidity	Standard ERC20, no fee-on-transfer, no rebasing, returns bool; centrally administered USDC	Non-standard behavior breaks fee/payout accounting; issuer freeze/ blacklist of vault, seller, or treasury blocks redemptions and payouts; transferFrom allowance front-run

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
			Standard ERC20, no fee-on-transfer, no rebasing, returns bool; centrally administered USDC	Non-standard behavior breaks fee/payout accounting; issuer freeze/ blacklist of vault, seller, or treasury blocks redemptions and payouts; transferFrom allowance front-run
priceSigner — off-chain backend signer	External EOA / off-chain service	Signs EIP-712 BuyPrice(tokenId , lienPrice, maturity, deadline, baseValue, feeVersion) consumed by buyNFT	Backend signs correct, non-manipulable prices and capital-gains base values; signing key kept secret; deadlines short	Key compromise → buy at attacker-chosen price / evade capital-gains fee; stale signature replay across deadline window; backend outage halts all buys
OPERATOR_ROLE backend (redeemLien caller)	Semi-trusted Role	Supplies currentRedemptiveValue and triggers lien redemption payouts and burns	Operator computes the correct redemptive value off-chain and is not malicious	Wrong or malicious value over-/under-pays the holder or drains vault liquidity; no on-chain bound check on redemptive value

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Lien certificate / real-world tax lien	Off-chain real-world asset	The legal asset the NFT represents; metadata pinned via IPFS hash	Off-chain legal validity, accurate metadata, correct maturity at mint	Off-chain/on-chain data mismatch; IPFS unavailability; no on-chain enforcement of real-world state enforcement of real-world state
Treasury wallet (FeeManager.treasury)	External EOA / Multisig	Receives sale fee, capital-gains fee, seller surcharge, and interest / pre-sale profit-sharing fee	Treasury is protocol-controlled and able to receive ERC20	Misconfigured treasury misroutes all fees; address(0) treasury reverts settlement (handled defensively)

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

LienNFT.SOL

Contract Name	Function	Category
LienNFT.sol	constructor()	**What this function can do** Disable initializers on the implementation contract **What this function cannot/should not do** Initialize state, be called on the proxy

Contract Name	Function	Category
LienNFT.sol	initialize(address initialOwner)	**Main invariant(s)** Implementation is never initializable directly **Access level** Implicit restriction (once at deployment) **What this function can do** Set ERC721 name/symbol, grant DEFAULT_ADMIN_ROLE and MINTER_ROLE to initialOwner, set nextTokenId = 1 **What this function cannot/should not do** Be called twice, accept zero owner **Main invariant(s)** Runs exactly once; admin/minter bootstrapped to a non-zero address **Access level** initializer (one-time)
LienNFT.sol	transferOwnership(address newOwner)	**What this function can do** Grant DEFAULT_ADMIN_ROLE to newOwner and revoke it from caller **What this function cannot/should not do** Set zero owner; leave protocol admin-less unintentionally **Main invariant(s)** Exactly one admin transition; non-zero target **Access level** DEFAULT_ADMIN_ROLE

Contract Name	Function	Category
LienNFT.sol	initialize(address initialOwner)	**Main invariant(s)** Implementation is never initializable directly **Access level** Implicit restriction (once at deployment) **What this function can do** Set ERC721 name/symbol, grant DEFAULT_ADMIN_ROLE and MINTER_ROLE to initialOwner, set nextTokenId = 1 **What this function cannot/should not do** Be called twice, accept zero owner **Main invariant(s)** Runs exactly once; admin/minter bootstrapped to a non-zero address **Access level** initializer (one-time)
LienNFT.sol	transferOwnership(address newOwner)	**What this function can do** Grant DEFAULT_ADMIN_ROLE to newOwner and revoke it from caller **What this function cannot/should not do** Set zero owner; leave protocol admin-less unintentionally **Main invariant(s)** Exactly one admin transition; non-zero target **Access level** DEFAULT_ADMIN_ROLE

Contract Name	Function	Category
LienNFT.sol	setLienVault(address vault)	**What this function can do** Set authorized vault, revoke VAULT_ROLE from old vault, grant to new **What this function cannot/should not do** Set zero vault; leave a stale vault with VAULT_ROLE **Main invariant(s)** Only the current vault holds VAULT_ROLE **Access level** DEFAULT_ADMIN_ROLE
LienNFT.sol	addMinter(address account) / removeMinter (address account)	**What this function can do** Grant / revoke MINTER_ROLE **What this function cannot/should not do** Grant minter to zero address; allow non-admin to mint authority **Main invariant(s)** Only admin controls the minter set **Access level** DEFAULT_ADMIN_ROLE
LienNFT.sol	mintLien(address recipient, LienDetails lienData)	**What this function can do** Mint a new lien NFT with metadata, increment nextTokenId, store details **What this function cannot/should not do** Mint with a duplicate id; mint liens for off-chain data that does not exist **Main invariant(s)** Token ids are unique and monotonic; details stored at mint **Access level** MINTER_ROLE

Contract Name	Function	Category
LienNFT.sol	bulkMintLiens (address recipient, LienDetails[] liens)	**What this function can do** Mint up to 100 liens to one recipient in a batch **What this function cannot/should not do** Mint to zero recipient; exceed batch cap of 100 **Main invariant(s)** 1 <= length <= 100; each minted id unique **Access level** MINTER_ROLE
LienNFT.sol	updateTokenURI(uint256 tokenId, string newURI) / bulkUpdateTokenURIs(...)	**What this function can do** Replace metadata URI for existing tokens **What this function cannot/should not do** Update non-existent token; mutate financial fields **Main invariant(s)** Token must exist; arrays length-matched in bulk **Access level** DEFAULT_ADMIN_ROLE
LienNFT.sol	updateRedemptionValue(uint256 tokenId, uint128 newValue)	**What this function can do** Overwrite RedemptionValue for an existing token **What this function cannot/should not do** Be called outside redemption flow; update non-existent token **Main invariant(s)** Only vault mutates redemption value; token must exist **Access level** VAULT_ROLE

Contract Name	Function	Category
LienNFT.sol	setRedemptionType(uint256 tokenId, string RedemptionType)	**What this function can do** Set "Early"/"Expired" redemption marker **What this function cannot/should not do** Be set by anyone but the vault; flip a redeemed lien back **Main invariant(s)** Redemption type set once at settlement **Access level** VAULT_ROLE
LienNFT.sol	setListedStatus(uint256 tokenId, bool status)	**What this function can do** Toggle the on-chain isListed flag **What this function cannot/should not do** Be toggled by non-vault callers; desync from real custody **Main invariant(s)** isListed reflects vault custody state **Access level** VAULT_ROLE
LienNFT.sol	burnLien(uint256 tokenId)	**What this function can do** Burn a lien NFT after redemption settlement **What this function cannot/should not do** Burn an NFT that is not owned/redeemed; burn outside redemption **Main invariant(s)** Burn only on a settled lien; token must exist **Access level** VAULT_ROLE

Contract Name	Function	Category
LienNFT.sol	lienDetails / getBulkLienDetails / getRedemptionValue / ownerOfSafe / supportsInterface	**What this function can do** Read-only views of lien data, owner (non-reverting), interface support **What this function cannot/should not do** Mutate state **Main invariant(s)** No state change; ownerOfSafe returns address(0) for nonexistent **Access level** Public view
LienNFT.sol	(inherited ERC721) transferFrom / safeTransferFrom / approve / setApprovalForAll	**What this function can do** Standard ERC721 transfer and approval of a held lien NFT **What this function cannot/should not do** Move an NFT that is locked in the vault (vault must hold it); bypass marketplace listing rules **Main invariant(s)** Standard ERC721 ownership/approval semantics; no transfer restriction overlay **Access level** Public (owner / approved)
LienNFT.sol	_authorizeUpgrade(address newImplementation)	**What this function can do** Authorize a UUPS implementation upgrade **What this function cannot/should not do** Permit non-admin upgrades; introduce storage-incompatible logic **Main invariant(s)** Only admin can change the implementation

Contract Name	Function	Category
		Access level DEFAULT_ADMIN_ROLE (internal)

LienVault.sol

Contract Name	Function	Category
LienVault.sol	constructor()	**What this function can do** Disable initializers on the implementation contract **What this function cannot/should not do** Initialize state **Main invariant(s)** Implementation never initializable directly **Access level** Implicit restriction (once at deployment)
LienVault.sol	initialize(...)	**What this function can do** Wire LienNFT and payment token; grant DEFAULT_ADMIN_ROLE, GUARDIAN_ROLE, OPERATOR_ROLE **What this function cannot/should not do** Run twice; accept zero addresses **Main invariant(s)** One-time bootstrap; all wired addresses non-zero **Access level** initializer (one-time)

Contract Name	Function	Category
LienVault.sol	transferOwnership(address newOwner)	**What this function can do** Move DEFAULT_ADMIN_ROLE to a new address **What this function cannot/should not do** Set zero owner; orphan admin **Main invariant(s)** Single admin transition; non-zero target **Access level** DEFAULT_ADMIN_ROLE
LienVault.sol	pause() / unpause()	**What this function can do** Halt / resume vault state-changing flows (lock, unlock, redeem) **What this function cannot/should not do** Unpause by guardian (admin only); block admin liquidity functions **Main invariant(s)** Pause stops user/redeem flows; only admin can resume **Access level** pause: GUARDIAN_ROLE or DEFAULT_ADMIN_ROLE; unpause: DEFAULT_ADMIN_ROLE
LienVault.sol	setMarketplace(address _marketplace)	**What this function can do** Set marketplace, revoke MARKETPLACE_ROLE from old, grant to new **What this function cannot/should not do** Set zero; leave stale marketplace with the role

Contract Name	Function	Category
LienVault.sol	setFeeManager(address _feeManager)	**Main invariant(s)** Only the current marketplace holds MARKETPLACE_ROLE **Access level** DEFAULT_ADMIN_ROLE **What this function can do** Point the vault at a fee-manager contract **What this function cannot/should not do** Set zero fee manager **Main invariant(s)** Fee manager non-zero when fees are computed **Access level** DEFAULT_ADMIN_ROLE
LienVault.sol	setFeeVersion(uint256 tokenId, uint256 feeVersion)	**What this function can do** Snapshot the fee version for a listed lien **What this function cannot/should not do** Set version 0; set for an unlisted lien **Main invariant(s)** feeVersion != 0; lien must be listed **Access level** MARKETPLACE_ROLE
LienVault.sol	setPurchasePrice(uint256 tokenId, uint256 purchasePrice)	**What this function can do** Record cost basis and mark hasPurchasePrice for the current holder **What this function cannot/should not do** Be set by non-marketplace callers; understate cost basis to inflate capital-gains fee

Contract Name	Function	Category
		Main invariant(s) Cost basis reflects the price actually paid at purchase **Access level** MARKETPLACE_ROLE
LienVault.sol	forceFeeVersion(uint256 tokenId, uint256 feeVersion, string reason)	**What this function can do** Emergency override of a lien's fee-version snapshot **What this function cannot/should not do** Set version 0; be used to retroactively change applicable fees against a holder **Main invariant(s)** feeVersion != 0; emitted with reason for audit trail **Access level** DEFAULT_ADMIN_ROLE
LienVault.sol	lockNFT(uint256 tokenId, address owner)	**What this function can do** Pull the lien NFT from the owner into the vault and mark listed **What this function cannot/should not do** Lock an NFT not owned by owner; run while paused **Main invariant(s)** Custody and isListed move together; owner check enforced **Access level** MARKETPLACE_ROLE

Contract Name	Function	Category
LienVault.sol	unlockNFT(uint256 tokenId, address newOwner)	**What this function can do** Release a vault-held NFT to newOwner and clear listed flag **What this function cannot/should not do** Release an NFT the vault does not hold; run while paused **Main invariant(s)** Vault must be the current owner before release **Access level** MARKETPLACE_ROLE
LienVault.sol	redeemLien(uint256 tokenId, uint128 currentRedemptiveValue)	**What this function can do** Settle a lien (early or expired, pre-sale or purchased): compute platform fee, pay holder/seller, pay treasury fee, pull/burn NFT **What this function cannot/should not do** Pay more than vault USDC balance; redeem an already-redeemed lien; bypass approval when NFT is owner-held **Main invariant(s)** ownerPayout = currentRedemptiveValue - platformFee; redeem-once (RedemptionType empty pre-call); balance ⩾ value; correct cost-basis/fee path per pre-sale vs purchased **Access level** OPERATOR_ROLE or DEFAULT_ADMIN_ROL

Contract Name	Function	Category
LienVault.sol	addLiquidity(uint256 amount)	**What this function can do** Pull USDC from caller into the vault redemption pool **What this function cannot/should not do** Accept zero; mint any receipt/share or grant a withdrawal right **Main invariant(s)** amount > 0; deposit is a non-refundable contribution (no depositor accounting) **Access level** Public
LienVault.sol	removeLiquidity(uint256 amount, address to)	**What this function can do** Withdraw a specific USDC amount from the vault to to **What this function cannot/should not do** Send to zero; withdraw more than balance; strand pending-redemption funds **Main invariant(s)** to != 0, amount > 0, balance >= amount **Access level** DEFAULT_ADMIN_ROLE
LienVault.sol	removeFullLiquidity(address to)	**What this function can do** Sweep the entire vault USDC balance to to **What this function cannot/should not do** Send to zero; be used while redemptions are outstanding without operational care **Main invariant(s)** to != 0; full-balance transfer (centralization risk)

Contract Name	Function	Category
LienVault.sol	_authorizeUpgrade(address newImplementation)	**Access level** DEFAULT_ADMIN_ROLE **What this function can do** Authorize UUPS implementation upgrade **What this function cannot/should not do** Permit non-admin upgrades; break storage layout / __gap **Main invariant(s)** Only admin upgrades; storage-compatible logic **Access level** DEFAULT_ADMIN_ROLE (internal)

LienMarketplace.sol

Contract Name	Function	Category
LienMarketplace.sol	constructor()	**What this function can do** Disable initializers on the implementation **What this function cannot/should not do** Initialize state **Main invariant(s)** Implementation never initializable directly **Access level** Implicit restriction (once at deployment)
LienMarketplace.sol	initialize(...)	**What this function can do** Wire LienNFT, LienVault, payment token, price signer; grant DEFAULT_ADMIN_ROLE, GUARDIAN_ROLE, OPERATOR_ROLE, VAULT_ROLE; init EIP-712 domain

Contract Name	Function	Category
		What this function cannot/should not do Run twice; accept zero addresses; accept a non-contract payment token **Main invariant(s)** One-time bootstrap; payment token is a contract; roles wired correctly **Access level** initializer (one-time)
LienMarketplace.sol	transferOwnership(address newOwner)	**What this function can do** Move DEFAULT_ADMIN_ROLE to a new address **What this function cannot/should not do** Set zero owner; orphan admin **Main invariant(s)** Single admin transition; non-zero target **Access level** DEFAULT_ADMIN_ROLE
LienMarketplace.sol	pause() / unpause()	**What this function can do** Halt / resume list, buy, cancel flows **What this function cannot/should not do** Unpause by guardian (admin only) **Main invariant(s)** Pause stops user trading; only admin resumes **Access level** pause: GUARDIAN_ROLE or DEFAULT_ADMIN_ROLE; unpause: DEFAULT_ADMIN_ROLE

Contract Name	Function	Category
LienMarketplace.sol	setPriceSigner(address newSigner)	**What this function can do** Rotate the authorized backend price signer **What this function cannot/should not do** Set zero signer **Main invariant(s)** priceSigner != 0; rotation emits old→new **Access level** OPERATOR_ROLE or DEFAULT_ADMIN_ROL
LienMarketplace.sol	setFeeManager(address newFeeManager)	**What this function can do** Point the marketplace at a fee-manager contract **What this function cannot/should not do** Set zero fee manager **Main invariant(s)** Fee manager non-zero before list/buy **Access level** DEFAULT_ADMIN_ROLE
LienMarketplace.sol	setSellerSurcharge(uint256 tokenId, uint256 surcharge)	**What this function can do** Set/override the per-listing buyer surcharge **What this function cannot/should not do** Be set by non-admin; silently change a buyer's total without their signed agreement **Main invariant(s)** Surcharge stored per listing and routed to treasury at buy **Access level** DEFAULT_ADMIN_ROLE

Contract Name	Function	Category
LienMarketplace.sol	listNFT(uint256 tokenId, uint256 sellerSurcharge)	**What this function can do** Lock caller's NFT in the vault, snapshot fee version, create an active listing (price 0, set at purchase) **What this function cannot/should not do** List a non-owned NFT; double-list; list when fee version is 0 / fee manager unset; run while paused **Main invariant(s)** Caller is NFT owner; not already listed; valid current fee version snapshotted **Access level** Public (NFT owner)
LienMarketplace.sol	buyNFT(tokenId, lienPrice, maturity, deadline, baseValue, feeVersion, signature)	**What this function can do** Verify backend EIP-712 price signature, enforce fee-version match and maturity, collect sale + capital-gains fees + surcharge, pay seller, unlock NFT to buyer, record cost basis **What this function cannot/should not do** Accept expired/replayed/forged signature; accept a fee version != current; mismatched maturity; underpay treasury/seller; run while paused **Main invariant(s)** totalAmount = lienPrice + sellerSurcharge; sellerProceeds = totalAmount - platformFee - capitalGainsFee - sellerSurcharge; signature single-use; feeVersion == currentFeeVersion; signed maturity == on-chain maturity **Access level** Public (payment via buyer's USDC allowance)

Contract Name	Function	Category
LienMarketplace.sol	cancelListing(uint256 tokenId)	**What this function can do** Cancel an active listing and return the NFT to the original seller **What this function cannot/should not do** Be called by anyone but the seller; cancel an inactive listing; run while paused **Main invariant(s)** Only the seller cancels; listing deactivated and NFT returned atomically **Access level** Public (listing seller)
LienMarketplace.sol	adminCancel Listing(uint25 6 tokenId)	**What this function can do** Mark a listing inactive on behalf of the vault during redemption **What this function cannot/should not do** Be called by anyone but the vault; move the NFT itself **Main invariant(s)** Only VAULT_ROLE; pure listing-state flag flip used inside redemption settleme **Access level** VAULT_ROLE
LienMarketplace.sol	getListingSell er / getListingSell erRaw	**What this function can do** Return the seller for an active listing (raw, or only when a purchase price exists) **What this function cannot/should not do** Mutate state; return a seller for an inactive listing **Main invariant(s)** Returns address(0) when listing is inactive (raw) or also when no purchase history (non-raw)

Contract Name	Function	Category
LienMarketplace.sol	_authorizeUpgrade(address newImplementation)	**Access level** Public view **What this function can do** Authorize UUPS implementation upgrade **What this function cannot/should not do** Permit non-admin upgrades; break storage layout / __gap **Main invariant(s)** Only admin upgrades; storage-compatible logic **Access level** DEFAULT_ADMIN_ROLE (internal)

FeeManager.sol

Contract Name	Function	Category
FeeManager.sol	constructor(address initialOwner, address initialTreasury)	**What this function can do** Set the owner and the initial treasury **What this function cannot/should not do** Accept zero treasury **Main invariant(s)** Treasury non-zero at deployment; currentFeeVersion starts at 0 (no config until first create) **Access level** Implicit restriction (once at deployment)

Contract Name	Function	Category
FeeManager.sol	createFeeConfig(FeeConfig config)	**What this function can do** Validate bps bounds, store a new versioned fee config, hash it, increment currentFeeVersion **What this function cannot/should not do** Store a config with any bps > 10000; mutate an existing version in place **Main invariant(s)** Each bps field ⩽ BPS_DENOMINATOR (10000); versions are append-only and monotonic; hash recorded per version **Access level** onlyOwner
FeeManager.sol	updateTreasury(address newTreasury)	**What this function can do** Change the treasury that receives all fees **What this function cannot/should not do** Set zero treasury **Main invariant(s)** Treasury always non-zero; change emits old→new **Access level** onlyOwner
FeeManager.sol	getFeeConfig(uint256 version) / feeScheduleHash(uint256 version)	**What this function can do** Return the stored config / hash for a known version **What this function cannot/should not do** Return data for an unknown version (reverts FeeConfigNotFound)

Contract Name	Function	Category
FeeManager.sol	(Ownable2Step) transferOwnership / acceptOwnership / renounceOwnership	**Main invariant(s)** Only versions with a recorded hash are readable **Access level** Public view **What this function can do** Two-step ownership handoff of fee/treasury authority **What this function cannot/should not do** Transfer to an address that has not accepted; renounce leaving fees unmanageable (operationally discouraged) **Main invariant(s)** Pending owner must explicitly accept before owner changes **Access level** owner (transfer/renounce); pending owner (accept)

LienfiUUPSProxy.sol

Contract Name	Function	Category
LienfiUUPSProxy.sol	constructor(address implementation, bytes data)	**What this function can do** Deploy an ERC1967 proxy pointed at implementation and delegatecall data (the initialize call) **What this function cannot/should not do** Point at a non-contract / uninitialized implementation; be re-pointed except via the implementation's _authorizeUpgrade

Contract Name	Function	Category
		Main invariant(s) Implementation set once at construction; subsequent upgrades only through UUPS admin path **Access level** Implicit restriction (once at deployment)

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
LienVault.sol	ERC20 (USDC)	Redemption liquidity pool — deposits from addLiquidity, source of all redemption payouts and redemption-time treasury fees
LienVault.sol	ERC721 (Lien NFT)	Lien NFTs held while listed on the marketplace and transiently during redemption settlement before burn

Note: LienMarketplace.sol does not custody value sale payments and fees are routed by direct transferFrom from the buyer to seller/treasury within buyNFT, with no escrow. FeeManager.sol holds no funds (treasury is an external wallet). LienNFT.sol holds metadata/state, not value.

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LienVault.sol	addLiquidity	ERC20 (USDC)	–	Public	Unaccounted donation: no receipt/share token and no depositor withdrawal right; only admin can ever remove it
LienVault.sol	removeLiquidity	–	ERC20 (USDC)	Admin	Centralization — admin can withdraw redemption liquidity at any time
LienVault.sol	removeFullLiquidity	–	ERC20 (USDC)	Admin	Sweeps entire balance, including funds earmarked for outstanding redemptions
LienVault.sol	redeemLien	ERC721 (pulled in / burned)	ERC20 payout to holder + ERC20 fee to treasury	Operator / Admin	Operator-supplied currentRedemptiveValue with no on-chain upper bound; cost-basis & fee-path selection (pre-sale vs purchased); external NFT + token calls under nonReentrant; payout reverts if USDC balance insufficient

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LienVault.sol	lockNFT	ERC721 (Lien NFT)	–	Marketplace	NFT pulled from owner into vault on listing; owner-equality check enforced
LienVault.sol	unlockNFT	–	ERC721 (Lien NFT)	Marketplace	NFT released to buyer (on sale) or seller (on cancel); requires vault to currently own it
LienMarketplace.sol	listNFT	ERC721 (→ vault)	–	Public (NFT owner)	Triggers vault lockNFT + fee-version snapshot; reentrancy guarded
LienMarketplace.sol	buyNFT	ERC20 (buyer → seller & treasury via transferFrom)	ERC721 (→ buyer, via vault unlock)	Public	Backend-signed price + fee-version match + maturity check; surcharge and capital-gains base value; no escrow (relies on buyer USDC allowance); signature single-use within deadline
LienMarketplace.sol	cancelListing	—	ERC721 (→ seller)	Public (listing seller)	Returns NFT to seller via vault unlockNFT; seller-only, listing must be active

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LienNFT.sol	mintLien / bulkMintLiens	–	ERC721 (mint, ⩽100/ batch)	Minter	Mints lien NFTs trusting off-chain lien data; batch capped at 100
LienNFT.sol	burnLien	ERC721 (burned)	–	Vault	Burns the NFT after redemption settlement; vault-only, token must exis

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of LienFi. We performed our audit according to the procedure described above.

Issues of High, Medium, Low and Informational Severity were found during Audit.

Few of the issues were acknowledged and others were resolved.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

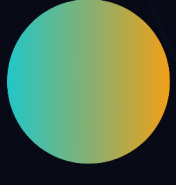
8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

June 2026

For



LienFi

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com